



AT6010 SOC HCI Protocol

Version 1.0

Table of Contents

1 概述	4
2 通信协议定义	4
2.1 发送帧数据格式	4
2.2 回复帧数据格式	4
3 命令说明	5
3.1 基本命令	5
3.1.1 设置感应电平持续时间	5
3.1.2 获取感应电平持续时间	5
3.1.3 设置光敏阈值	6
3.1.4 获取光敏阈值	6
3.1.5 保存雷达设置	6
3.1.6 获取雷达保存状态	7
3.1.7 设置 OUT 信号状态	7
3.1.8 设置 PWM 占空比	7
3.1.9 System Reset	8
3.1.10 获取软硬件版本号	8
3.1.11 波特率切换	9
3.2 雷达配置命令	9
3.2.1 设置雷达感应功能	9
3.2.2 获取雷达感应状态	10
3.2.3 获取当前算法类型	10
3.2.4 设置雷达感应等级	10
3.2.5 获取雷达感应等级	10
3.2.6 获取雷达感应信息	11
3.2.7 获取算法配置的边界值	12
3.2.8 获取算法感应配置	12
3.2.9 设置运动检测最远感应距离	13
3.2.10 设置运动检测最近感应距离	14
3.2.11 设置运动检测灵敏度	14

3.2.12 设置微动检测最远感应距离	14
3.2.13 设置微动检测最近感应距离	15
3.2.14 设置微动检测灵敏度	15
3.2.15 设置呼吸检测最远感应距离	15
3.2.16 设置呼吸检测最近感应距离	16
3.2.17 设置呼吸检测灵敏度	16
3.2.18 获取雷达测高信息	16
3.3 超低功耗(ULP)模式命令	17
3.3.1 设置非低功耗运行时间	17
3.3.2 读 AT5815C 寄存器命令	17
3.3.3 写 AT5815C 寄存器命令	18
3.4 主动上报命令	18
3.4.1 上报完整检测信息	20
3.4.2 上报测高仪检测信息	20
3.4.3 上报占位检测信息	21
3.4.4 上报运动存在检测信息	21
3.4.5 上报呼吸心率检测信息	22
3.4.6 上报分区检测信息	23
3.5 调试命令	23
3.5.1 Register Write	23
3.5.2 Register Read	24
3.5.3 Memory Write	24
3.5.4 Memory Read	24
3.5.5 Flash Write	25
4 硬件接口	25
4.1 UART 接口	25
4.2 IIC 接口	25
4.2.1 IIC 时序	25
Revision History	26

1 概述

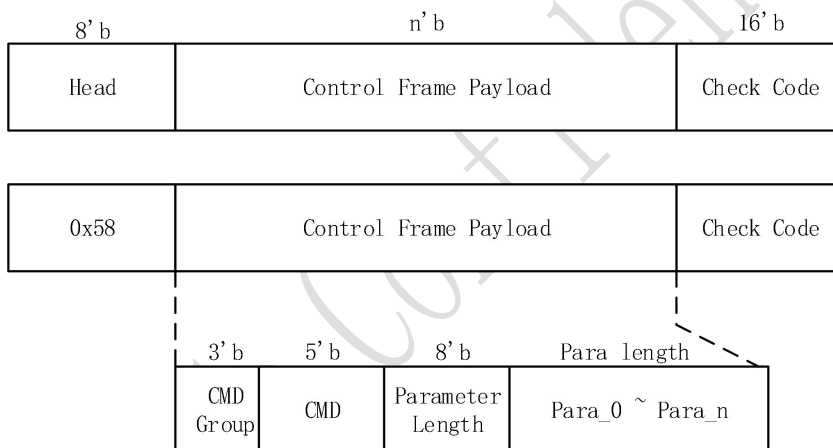
隔空科技自主研发的 AT6010 60GHz 雷达 SOC 芯片支持通过 UART 接口和上位机进行通信，包括基本功能命令，FMCW 配置命令和调试指令等。本文主要介绍了 AT6010 的通信协议格式和各命令说明。

2 通信协议定义

通信协议的帧数据主要分为发送帧数据和回复帧数据，发送帧为上位机给 AT6010 SOC 发送控制命令的帧数据格式，回复帧为 AT6010 SOC 执行完上位机控制的指令后回复给上位机的帧数据格式。详细的帧数据格式描述如下。

2.1 发送帧数据格式

发送帧主要为上位机通过 UART 发送给 AT6010 芯片的帧数据，其格式定义如下：



Description:

Head: 发送帧帧头，值为 0x58；

Control Frame Payload: 发送帧有效负荷，分命令分组，控制命令，参数长度和参数：

CMD Group: 命令分组，3 个有效位，最多支持 7 个分组的命令；

CMD: 控制命令，5 个有效位，即一个分组最多支持 32 条控制指令；

Parameter Length: 参数的总长度（字节）；

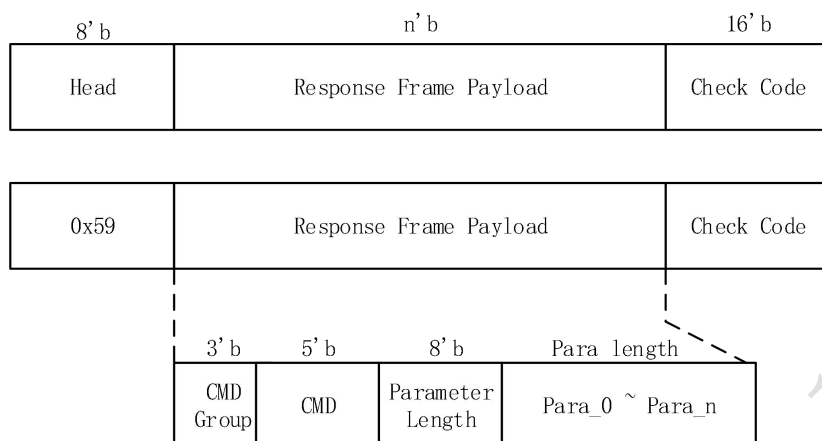
Para_0 ~ Para_n: 参数（长度由 Parameter Length 指定）

Check Code: 校验码，值为前面数据的和：

$Check\ Code = Head + Payload_0 + \dots + Payload_n$

2.2 回复帧数据格式

SOC 芯片执行控制命令后的回复数据，回复数据帧格式定义如下：



Description:

Head: 回复帧帧头，值为 0x59;

Response Frame Payload: 回复帧有效负荷，分命令分组，控制命令，参数长度和参数:

CMD Group: 回复对应的执行命令分组;

CMD: 回复对应的执行命令;

Parameter Length: 回复参数的总长度 (字节);

Para_0 ~ Para_n: 参数 (长度由 Parameter Length 指定)

Check Code: 校验码，值为前面数据的和;

Check Code = Head + Payload_0 + ... + Payload_n

3 命令说明

3.1 基本命令

3.1.1 设置感应电平持续时间

功能说明:

当雷达感应到运动物体时，OUT PIN 信号输出有效电平持续时间，若 OUT 配置为 PWM 模式时，则为输出 PWM 持续时间。

指令码: 0x4

参数: para1: time(low byte), para2: time(high byte); In seconds

发送命令格式 (HEX): 58 04 02 Byte0 Byte1 xx xx (xx xx 表示发送帧/回复帧校验码,以下同理)

回复帧: 参考回复帧格式

e.g., set time to 1s

Send: 58 04 02 01 00 5F 00

Resp: 59 04 01 00 5E 00 (success). Other: fail.

3.1.2 获取感应电平持续时间

功能说明:

获取雷达芯片/模块雷达感应的有效电平持续时间。

指令码: 0x5

参数: none

发送命令格式 (HEX): 58 05 00 5D 00

回复帧: 参考回复帧格式

e.g. get lot, the result value is 0x0001 (1s)

Send: 58 05 00 5D 00

Resp: 59 05 02 01 00 61 00

3.1.3 设置光敏阈值

功能说明:

设置雷达芯片/模块光敏感应的阈值, 只有硬件支持光敏并且软件打开该功能时有效。

指令码: 0x6

参数: para1: lux(low byte), para2: lux(high byte)

发送命令格式 (HEX): 58 06 02 Byte0 Byte1 xx xx

回复帧: 参考回复帧格式

e.g., set lux to 1000

Send: 58 06 02 E8 03 4B 01

Resp: 59 06 01 00 60 00 (success). Other: fail.

3.1.4 获取光敏阈值

功能说明:

获取雷达芯片/模块光敏感应的阈值, 只有硬件支持光敏并且软件打开该功能时有效。

指令码: 0x7

参数: none

发送命令格式 (HEX): 58 07 00 5F 00

回复帧: 参考回复帧格式

e.g., get lux, the result value is 0x03E8

Send: 58 07 00 5F 00

Resp: 59 07 02 E8 03 4D 01

3.1.5 保存雷达设置

功能说明:

主要将设置的参数 (感应距离、OUT 信号持续时间、光敏阈值、雷达打开/关闭状态和波特率) 保存在模块的 Flash 中, 重新开机将加载上次设置的参数值。由于命令交互和芯片内部 flash 写入时间较长, 如果需要将参数保存在雷达模块中, 建议发送命令后延迟 1 秒以

上再发送其它命令。

指令码: 0x8

参数: para1: 0x1 (保存), 0x0 (不保存)

发送命令格式 (HEX): 58 08 01 01 62 00 (保存)

58 08 01 00 61 00 (不保存)

回复帧: 59 08 01 00 62 00 (success). Other: fail.

3.1.6 获取雷达保存状态

功能说明:

获取设置的参数 (感应距离、OUT 信号持续时间和光敏阈值) 是否已保存。若设置参数但没有发送保存雷达设置命令而直接关机, 设置的参数在下次启动时不会生效。

指令码: 0x9

参数: none

发送命令格式 (HEX): 58 09 00 61 00

回复帧: 59 09 01 01 64 00 // e.g., save is on

59 09 01 00 63 00 // e.g., save is off

3.1.7 设置 OUT 信号状态

功能说明:

若 OUT 配置为 IO 时, 设置 OUT PIN 为高电平或低电平, 若 OUT 配置为 PWM 模式时, 则设置是否输出 PWM 信号。

指令码: 0xA

参数: para1: 0x1 (设置 OUT 有效), 0x0 (设置 OUT 无效)

发送命令格式 (HEX): 58 0A 01 01 64 00 (设置 OUT 有效)

58 0A 01 00 63 00 (设置 OUT 无效)

回复帧: 59 0A 01 00 64 00 (success). Other: fail

3.1.8 设置 PWM 占空比

功能说明:

当 OUT 配置为 PWM 模式时, 可通过该命令设置 PWM 输出的占空比。

指令码: 0xB

参数: para1: value of duty (low byte), para2: value of duty (high byte). Unit 0.1%

发送命令格式 (HEX): 58 0B 02 Byte0 Byte1 xx xx

回复帧: 参考回复帧格式

e.g., set duty to 50.0%

Send: 58 0B 02 F4 01 5A 01

Resp: 59 0B 01 00 65 00 (success). Other: fail.

Note: 设置打开灯时 PWM 占空比。

3.1.9 System Reset

功能说明:

系统复位, 发送该命令后, 系统将重启复位。

指令码: 0x13

参数: para1: reset mode (uint8)

发送命令格式 (HEX): 58 13 01 01 6D 00

回复帧: 根据波特率不同, 回复的数据有所差异

59 13 01 00 6D 00 ...(success). Other: fail.

(...代表 uart0 在芯片 reset 过程中的数据输出)

3.1.10 获取软硬件版本号

指令码: 0xFE

参数: none

发送命令格式 (HEX): 58 FE 00 56 01

回复帧: 59 FE 08 Byte0 Byte1 Byte2 ... Byte7 xx xx

```
1. typedef struct _ci_ver_struct {
2.     u8 sw_ver_major;
3.     u8 sw_ver_minor;
4.     u8 sw_ver_revision;
5.     u8 cust_ver_major;
6.     u8 cust_ver_minor;
7.     u8 hw_ver_major;
8.     u8 hw_ver_minor;
9.     u8 reserved;
10. } ci_ver_struct;
```

字段说明:

字段	功能说明
sw_ver_major	软件 SDK 版本号, 例如 v0.3.2, sw_ver_major = 0x00
sw_ver_minor	软件 SDK 版本号, 例如 v0.3.2, sw_ver_minor = 0x03
sw_ver_revision	软件 SDK 版本号, 例如 v0.3.2, sw_ver_revision = 0x02
cust_ver_major	用户自定义版本号, 例如 v1.2, cust_ver_major = 0x01
cust_ver_minor	用户自定义版本号, 例如 v1.2, cust_ver_minor = 0x02
hw_ver_major	硬件版本号, 例如 v1.2, hw_ver_major = 0x01
hw_ver_minor	硬件版本号, 例如 v1.2, hw_ver_minor = 0x02

reserved	保留
----------	----

3.1.11 波特率切换

功能说明：

默认波特率为 921600bps，可通过命令切换 9600bps、115200bps 或其他波特率。

指令码：0x19

参数：波特率数值，4 个字节表示，小端格式

发送命令格式（HEX）：58 19 04 Byte0 Byte1 Byte2 Byte3 xx xx

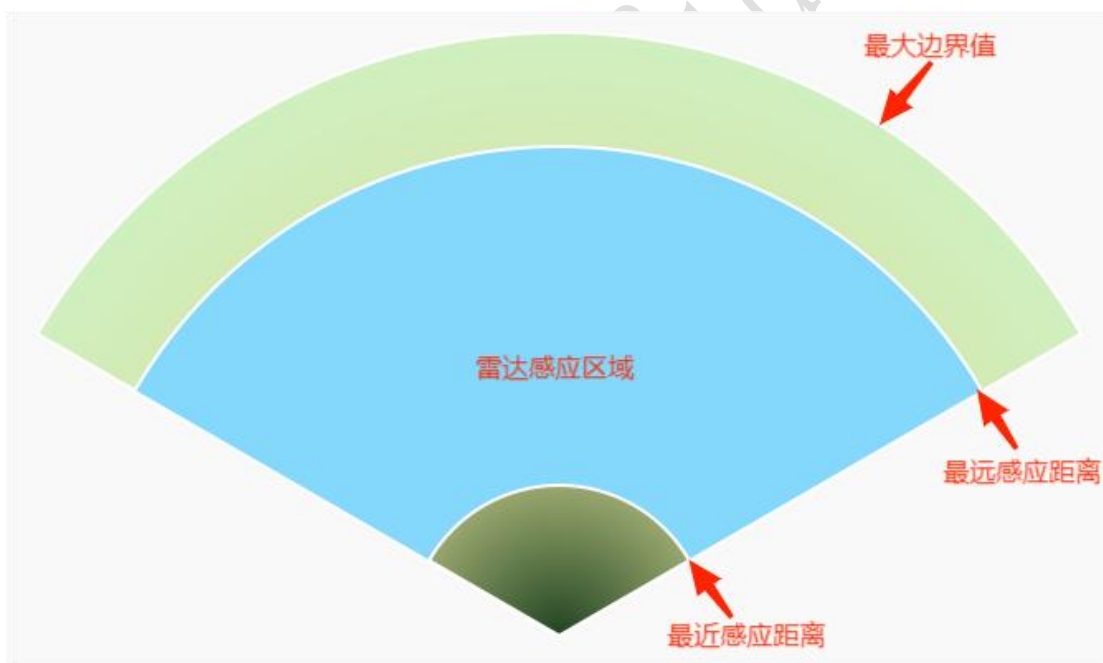
回复帧：参考回复帧格式

e.g., set baud rate to 115200bps

Send: 58 19 04 00 C2 01 00 38 01

Resp: 59 19 01 00 73 00 (success). Other: fail.

3.2 雷达配置命令



如上图，“最大边界值”是指雷达模块当前配置感应的极限范围，不可修改。“最近感应距离”和“最远感应距离”需用户根据实际应用场景进行配置，两者构成的蓝色区域是雷达模块可实际感应到并上报感应结果的范围，其他区域为雷达模块可实际感应到但不上报感应结果（即忽略掉）的范围。

3.2.1 设置雷达感应功能

功能说明：

设置雷达感应功能开关。

指令码：0xD1

参数: para1: 0x1 (打开雷达感应), 0x0 (关闭雷达感应)

发送命令格式 (HEX): 58 D1 01 01 2B 01 (打开雷达)

58 D1 01 00 2A 01 (关闭雷达)

回复帧: 59 D1 01 00 2B 01 (success). Other: fail.

3.2.2 获取雷达感应状态

功能说明:

获取当前雷达感应功能是否有效。

指令码: 0xD0

参数: none

发送命令格式 (HEX): 58 D0 00 28 01

回复帧: 59 D0 01 01 2B 01 // return 1: radar is on

59 D0 01 00 2A 01 // return 0: radar is off

3.2.3 获取当前算法类型

功能说明:

隔空发布的 SDK 支持多种算法类型, 可通过该命令获取雷达芯片/模块运行的算法类型。
(当前算法版本无效)

指令码: 0x31

参数: none

发送命令格式 (HEX): 58 31 00 89 00

回复帧: 59 31 01 Byte0 xx xx

3.2.4 设置雷达感应等级

功能说明:

设置雷达感应档位, 每个档位设置包括了雷达感应灵敏度和感应范围。

指令码: 0x2

参数: para1: distance level 0 - 15

发送命令格式 (HEX): 58 02 01 Byte0 xx xx

回复帧: 参考回复帧格式

e.g., set distance level to 15

Send: 58 02 01 0F 6A 00

Resp: 59 02 01 00 5C 00 (success). Other: fail.

3.2.5 获取雷达感应等级

功能说明:

获取雷达芯片/模块的感应等级。

指令码: 0x3

参数: none

发送命令格式 (HEX): 58 03 00 5B 00

回复帧: 参考回复帧格式

e.g., get distance level, the result value is 15

Send: 58 03 00 5B 00

Resp: 59 03 01 0F 6C 00

3.2.6 获取雷达感应信息

功能说明:

获取雷达感应信息, 包括运动、微动和存在感应信息。

指令码: 0x30

参数: none

发送命令格式 (HEX): 58 30 00 88 00

回复帧: 59 30 14 Byte0 Byte1 Byte2... Byte19 xx xx

```
1. typedef struct {
2.     u8 is_detected;    /* the detected result that combined motion, micro-motion and presence. */
3.     u8 det_result;     /* detection result */
4.     u16 range_val;     /* range detection value, unit: mm */
5.     s16 angle_val;     /* angle detection value, unit: 1degree */
6.     s16 velo_val;     /* velocity, todo. */
7.     u8 reserved[6];    /* reserved for extend. */
8.
9.     u8 rb_conf;        /* mix range detection confidence = 0~16, if confidence < 12, range_val may be incorrect */
10.    u8 angle_conf;     /* mix angle detection confidence = 0~16, if confidence < 8, angle_val may be incorrect */
11.    u32 frame_idx;     /* frame index. */
12. } fmcw_det_info_t;
```

字段说明:

字段	功能说明
is_detected	运动,微动和存在综合判定结果
det_result	算法检测状态: 0x01: 靠近 0x02: 远离

	0x04: 运动 0x08: 微动 0x10: 呼吸
range_val	检测距离值,单位: mm
angle_val	检测角度值,单位: 1degree
velo_val	检测速度值,目前预留
reserved[6]	预留待扩展
rb_conf	综合距离置信度,范围 0~16,若<12,距离值可能不准确
angle_conf	综合角度置信度,范围 0~16,若<8,距离值可能不准确
frame_idx	帧索引

3.2.7 获取算法配置的边界值

功能说明:

获取用户检测区域的边界值。

指令码: 0x32

参数: none

发送命令格式 (HEX): 58 32 00 8A 00

回复帧: 59 32 0E Byte0 Byte1 Byte2 ... Byte13 xx xx

```

1. typedef struct {
2.     u16 det_mot_min;
3.     u16 det_mot_max;
4.     u16 det_micro_min;
5.     u16 det_micro_max;
6.     u16 det_bhr_min;
7.     u16 det_bhr_max;
8.     u16 det_swp_bw;
9. } mdet_bound_cfg;

```

字段说明:

字段	功能说明
det_mot_min	雷达模块运动目标检测最小边界值,单位 cm,不可更改
det_mot_max	雷达模块运动目标检测最大边界值,单位 cm,不可更改
det_micro_min	雷达模块微动目标检测最小边界值,单位 cm,不可更改
det_micro_max	雷达模块微动目标检测最大边界值,单位 cm,不可更改
det_bhr_min	雷达模块呼吸目标检测最小边界值,单位 cm,不可更改
det_bhr_max	雷达模块呼吸目标检测最大边界值,单位 cm,不可更改
det_swp_bw	扫频带宽,获取值为 0 表示 1GHZ,其它值暂不支持

3.2.8 获取算法感应配置

功能说明:

获取用户配置的运动/微动/呼吸检测最远/最近感应距离和灵敏度。

指令码: 0x33

参数: none

发送命令格式 (HEX): 58 33 00 8B 00

回复帧: 59 33 12 Byte0 Byte1 Byte2 ... Byte17 xx xx

```
1. typedef struct {
2.     u16 mot_det_sta;
3.     u16 mot_det_sto;
4.     u8  mot_det_sl;
5.     u8  reserved0;
6.
7.     u16 micro_det_sta;
8.     u16 micro_det_sto;
9.     u8  micro_det_sl;
10.    u8  reserved1;
11.
12.    u16 bhr_det_sta;
13.    u16 bhr_det_sto;
14.    u8  bhr_det_sl;
15.    u8  reserved2;
16. } mdet_default_cfg;
```

字段说明:

字段	功能说明
mot_det_sta	雷达模块运动检测的最近感应距离,单位 cm
mot_det_sto	雷达模块运动检测的最远感应距离,单位 cm
mot_det_sl	雷达模块运动检测的灵敏度,范围[0~10],越小越灵敏
reserved0	保留
micro_det_sta	雷达模块微动检测的最近感应距离,单位 cm
micro_det_sto	雷达模块微动检测的最远感应距离,单位 cm
micro_det_sl	雷达模块微动检测的灵敏度,范围[0~10],越小越灵敏
reserved1	保留
bhr_det_sta	雷达模块呼吸检测的最近感应距离,单位 cm
bhr_det_sto	雷达模块呼吸检测的最远感应距离,单位 cm
bhr_det_sl	雷达模块呼吸检测的灵敏度,范围[0~10],越小越灵敏
reserved2	保留

3.2.9 设置运动检测最远感应距离**功能说明:**

设置当前芯片/模块运动检测最远感应距离，单位为 cm，最大为算法设置的边界值。设置成功后将覆盖初始化设置值。

指令码：0xD2

参数：16-bits, para1: delta(low byte), para2: delta(high byte)

发送命令格式（HEX）：58 D2 02 Byte0 Byte1 xx xx

回复帧：参考回复帧格式

e.g., set range_val as 0x012C

Send: 58 D2 02 2C 01 59 01

Resp: 59 D2 01 00 2C 01(success). Other: fail.

3.2.10 设置运动检测最近感应距离

功能说明：

设置当前芯片/模块运动检测最近感应距离，单位 cm。如设置值为 200，则表示 2 米范围内的距离感应无效。

指令码：0x34

参数：16-bits, para1: range_delta(low byte), para2: range_delta(high byte)

发送命令格式（HEX）：58 34 02 Byte0 Byte1 xx xx

回复帧：参考回复帧格式

e.g., set range_val as 0x0001

Send: 58 34 02 01 00 8F 00

Resp: 59 34 01 00 8E 00(success). Other: fail.

3.2.11 设置运动检测灵敏度

功能说明：

设置当前芯片/模块运动检测感应灵敏度，数值越低越灵敏度，参数范围：0 ~ 10。

指令码：0x35

参数：8-bits, para: level

发送命令格式（HEX）：58 35 01 Byte0 xx xx

回复帧：参考回复帧格式

e.g., set level to 1

Send: 58 35 01 01 8F 00

Resp: 59 35 01 00 8F 00(success). Other: fail.

3.2.12 设置微动检测最远感应距离

功能说明：

设置当前芯片/模块微动检测最远感应距离，单位为 cm，最大为算法设置的边界值。（当前算法版本无效）

指令码：0x36

参数：16-bits, para1: range_val(low byte), para2: range_val(high byte)

发送命令格式（HEX）：58 36 02 Byte0 Byte1 xx xx

回复帧：参考回复帧格式

e.g., set range_val as 0x0100

Send: 58 36 02 00 01 91 00

Resp: 59 36 01 00 90 00(success). Other: fail.

3.2.13 设置微动检测最近感应距离

功能说明：

设置当前芯片/模块微动检测最近感应距离，单位 cm。如设置值为 200，则表示 2 米范围内的距离感应无效。（当前算法版本无效）

指令码：0x37

参数：16-bits, para1: range_val(low byte), para2: range_val(high byte)

发送命令格式（HEX）：58 37 02 Byte0 Byte1 xx xx

回复帧：参考回复帧格式

e.g., set range_val as 0x0100

Send: 58 37 02 00 01 92 00

Resp: 59 37 01 00 91 00(success). Other: fail.

3.2.14 设置微动检测灵敏度

功能说明：

设置当前芯片/模块微动检测感应灵敏度，数值越低越灵敏度，参数范围：0~10。（当前算法版本无效）

指令码：0x38

参数：8-bits, para:level

发送命令格式（HEX）：58 38 01 Byte0 xx xx

回复帧：参考回复帧格式

e.g., set level to 1

Send: 58 38 01 01 92 00

Resp: 59 38 01 00 92 00(success). Other: fail.

3.2.15 设置呼吸检测最远感应距离

功能说明:

设置当前芯片/模块呼吸检测最远感应距离, 单位为 cm, 最大为算法设置的边界值。

指令码: 0x39

参数: 16-bits, para1: range_val(low byte), para2: range_val(high byte)

发送命令格式 (HEX): 58 39 02 Byte0 Byte1 xx xx

回复帧: 参考回复帧格式

e.g., set range_val as 0x0100

Send: 58 39 02 00 01 94 00

Resp: 59 39 01 00 93 00(success). Other: fail.

3.2.16 设置呼吸检测最近感应距离

功能说明:

设置当前芯片/模块呼吸检测最近感应距离, 单位 cm。如设置值为 200, 则表示 2 米范围内的距离感应无效。

指令码: 0x3A

参数: 16-bits, para1: range_val(low byte), para2: range_val(high byte)

发送命令格式 (HEX): 58 3A 02 Byte0 Byte1 xx xx

回复帧: 参考回复帧格式

e.g., set range_val as 0x0100

Send: 58 3A 02 00 01 95 00

Resp: 59 3A 01 00 94 00(success). Other: fail.

3.2.17 设置呼吸检测灵敏度

功能说明:

设置当前芯片/模块呼吸检测感应灵敏度, 数值越低越灵敏度, 参数范围: 0 ~ 10。

指令码: 0x3B

参数: 8-bits, para:level

发送命令格式 (HEX): 58 3B 01 Byte0 xx xx

回复帧: 参考回复帧格式

e.g., set level to 1

Send: 58 3B 01 01 95 00

Resp: 59 3B 01 00 95 00(success). Other: fail.

3.2.18 获取雷达测高信息

功能说明:

获取雷达测高信息。

指令码: 0x30

参数: none

发送命令格式 (HEX): 58 30 00 88 00

回复帧: 59 30 14 Byte0 Byte1 Byte2... Byte19 xx xx

```
1. typedef struct {
2.     u8 is_detected;      /* 测高数据是否有效*/
3.     u8 reserved0;        /* 预留待扩展 */
4.     u16 height;          /* 检测到身高值,单位: mm */
5.     u8 reserved[12];     /* 预留待扩展 */
6.     u32 frame_idx;       /* 帧索引 */
7. } fmcw_det_info_t;
```

字段说明:

字段	功能说明
is_detected	测高数据是否有效
reserved0	预留待扩展
height	检测到身高值,单位: mm
reserved[12]	预留待扩展
frame_idx	帧索引

3.3 超低功耗(ULP)模式命令

3.3.1 设置非低功耗运行时间

功能说明:

芯片上电后处在非低功耗模式持续时间默认为 10 秒, 下发此命令可以修改运行时间, 参数范围 [0, 255], 如果设置为 0 表示立刻进入低功耗程序。

指令码: 0x90

参数: para1: active time 0~255

发送命令格式 (HEX): 58 90 01 Byte0 xx xx

回复帧: 参考回复帧格式

e.g., set ulp active time to 60 seconds

Send: 58 90 01 3C 25 01

Resp: 59 90 01 00 EA 00 (success). Others: fail.

3.3.2 读 AT5815C 寄存器命令

功能说明:

读 AT5815C 寄存器, 寄存器地址和数值都是 8 比特。

指令码: 0x91

参数: para1: reg_addr

发送命令格式 (HEX): 58 91 01 Byte0 xx xx

回复帧: 参考回复帧格式

e.g., read register 0x4C

Send: 58 91 01 4C 36 01

Resp: 59 91 01 01 EC 00

3.3.3 写 AT5815C 寄存器命令

功能说明:

写 AT5815C 寄存器, 寄存器地址和数据都是 8 比特。

指令码: 0x92

参数: para1: reg_addr, para2: reg_value

发送命令格式 (HEX): 58 92 02 Byte0 Byte1 xx xx

回复帧: 参考回复帧格式

e.g., set register 0x4C to 0x02

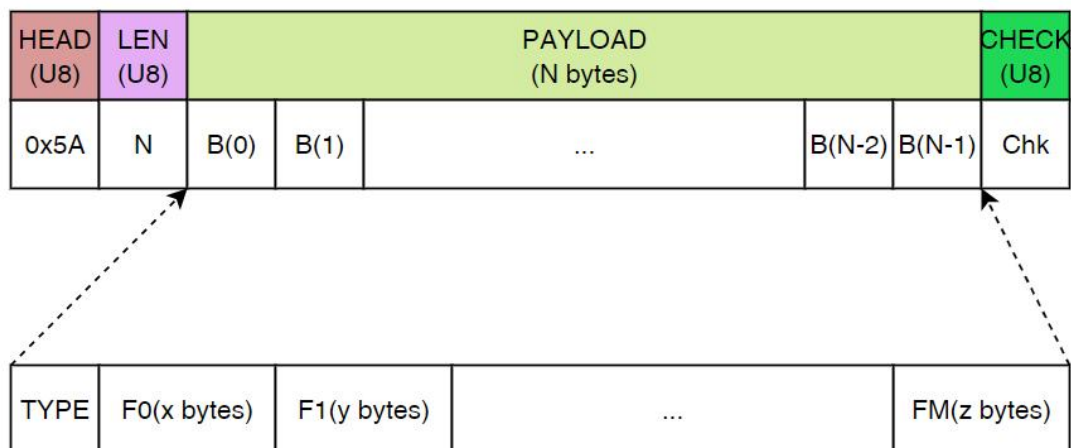
Send: 58 92 02 4C 02 3A 01

Resp: 59 92 01 00 EC 00(success). Others: fail.

3.4 主动上报命令

ULP 应用主动上报检测结果, 以数据流的形式上报, 格式和 CI 相似, 但有细微差别。其格式约定如下:

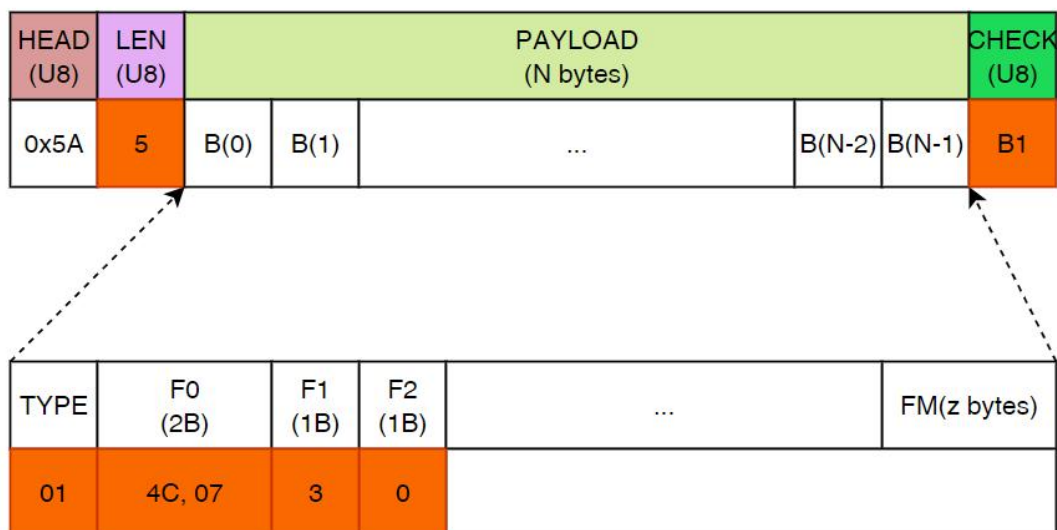
1. 所有字段都是 little-endian 格式;
2. HEAD 固定为 0x5A;
3. LEN 表示后面 PAYLOAD 字段占据字节个数;
4. CHECK 是 8 比特校验和, 计算内容包括: HEAD, LEN, PAYLOAD 三部分;
5. 每种 TYPE 对应的结构体不同, 其中 F0, F1, ..., FM 分别对应其结构体一个字段;



这里以 TYPE = 1 表示 HTM 结构体为例进行说明。

```
1. typedef struct {
2.     u16 htm;
3.     u8 status;
4.     u8 reserved;
5. } htm_det_info_t;
```

例如 htm = 1868 mm = 0x074C, status=3, reserved=0, 字节流如下图所示:



LEN = 1 + 2 + 2 = 5

F0 表示结构体中第一个字段, 这里表示 htm, 0x074C, 小端模式为 4C, 07;

F1 表示结构体中第二个字段, 这里表示 status, 0x03;

F2 表示结构体中第三个字段, 默认 reserved 一直为 0;

CHECK 校验和 = 5A + 05 + 01 + 4C + 07 + 03 = B6

注意: 本章的上报信息都是仅检测到目标才会输出, 未检测到不输出。

3.4.1 上报完整检测信息

对应的 **TYPE = 0**，结构体类型如下：

```
1. typedef struct {
2.     u8 is_detected; /* the detected result that combined motion, micro-motion and presence. */
3.     u8 det_result; /* detection result */
4.     u16 range_val; /* range detection value, unit: mm */
5.     s16 angle_val; /* angle detection value, unit: 1degree */
6.     s16 velo_val; /* velocity, todo. */
7.     u8 reserved[6]; /* reserved for extend. */
8.     u8 rb_conf; /* mix range detection confidence = 0~16, if confidence < 12, range_val may be incorrect */
9.     u8 angle_conf; /* mix angle detection confidence = 0~16, if confidence < 8, angle_val may be incorrect */
10.    u32 frame_idx; /* frame index. */
11. } fmcw_det_info_t;
```

字段说明：

字段	功能说明
is_detected	运动,微动和存在综合判定结果
det_result	算法检测状态: 0x01: 靠近 0x02: 远离 0x04: 运动 0x08: 微动 0x10: 呼吸
range_val	检测距离值,单位: mm
angle_val	检测角度值,单位: 1degree
velo_val	检测速度值,目前预留
reserved[6]	预留待扩展
rb_conf	综合距离置信度,范围 0~16,若<12,距离值可能不准确
angle_conf	综合角度置信度,范围 0~16,若<8,距离值可能不准确
frame_idx	帧索引

上报场景：需要上报所有检测信息。

3.4.2 上报测高仪检测信息

对应的 **TYPE = 1**，结构体类型如下：

```
1. typedef struct {
1.     u16 htm; /* height, mm */
```

```

2.    u8  status;          /* 0:invalid,1: calib done,2:enter htm, 3:htm done*/
3.    u8  reserved;
4.    } htm_det_info_t;

```

字段说明:

字段	功能说明
htm	身高,单位毫米,仅在 status 等于 3 有效
status	算法检测状态: 0x00: 无效 0x01: 到地高度校准完成 0x02: 进入测高区域 0x03: 测高已完成
reserved	预留待扩展

上报场景: 需要上报测高仪检测信息。

3.4.3 上报占位检测信息

对应的 **TYPE = 2**, 结构体类型如下:

```

1.  typedef struct {
2.      s8  rb;
3.      u8  pwr_db_raw; /* power raw */
4.      u8  pwr_db_rgc; /* power range compensation */
5.      u8  num_rb_greater_than_thr; /* num neighbor RB greater than threshold */
6.  } pod_det_info_t;

```

字段说明:

字段	功能说明
rb	Rb 编号
pwr_db_raw	原始能量值,单位 db
pwr_db_rgc	带距离补偿后的能量值,单位 db
num_rb_greater_than_thr	本 RB 之后的 refRbCount 之中,有多少个能量大于能量门限 threshold_DB

上报场景: 需要上报占位检测信息。

3.4.4 上报运动存在检测信息

对应的 **TYPE = 3**, 结构体类型如下:

```

1.  typedef struct {

```

```

2.    u8 is_detected;    /* the detected result that combined motion, micro-motion and presence. */
3.    u8 det_result;     /* detection result */
4.    u16 range_val;     /* range detection value, unit: mm */
5.    s16 angle_val;     /* angle detection value, unit: 1degree */
6.    s16 velo_val;      /* velocity, todo. */
7.    } mot_det_info_t;

```

字段说明:

字段	功能说明
is_detected	运动,微动和存在综合判定结果
det_result	算法检测状态: 0x01: 靠近 0x02: 远离 0x04: 运动 0x08: 微动 0x10: 呼吸
range_val	检测距离值,单位: mm
angle_val	检测角度值,单位: 1degree
velo_val	检测速度值,目前预留

上报场景: 需要上报运动存在检测信息。

3.4.5 上报呼吸心率检测信息

对应的 **TYPE = 4**, 结构体类型如下:

```

1.    typedef struct {
2.        u8 det_result;    /* detection result */
3.        u8 br_val;        /* breathing rate */
4.        u8 hr_val;        /* heart rate */
5.        u8 angle_val;     /* reserve */
6.        u16 range_val;    /* range val mm */
7.        u16 padding;      /* reserve */
8.    } bhr_det_info_t;

```

字段说明:

字段	功能说明
det_result	算法检测状态: 0x01 / 0x02 / 0x04 : 运动 0x08: 微动 0x10: 存在
br_val	呼吸频率值

hr_val	心率，心脏跳动的速率
angle_val	保留
range_val	检测距离值，单位：mm
padding	保留

上报场景：需要上报呼吸心率检测信息。

3.4.6 上报分区检测信息

对应的 **TYPE = 5**，结构体类型如下：

```

1. typedef struct {
2.     u32 obj_num;          /* Number of object,only output the nearest object
   in each region,max obj num = 3 */
3.     rgn_obj_info_t obj[3]; /* Detection information of the nearest object in
   each region, max region = 3 */
4. } rgn_det_info_t;
5.
6. typedef struct {
7.     u16 range_val;
8.     s16 angle_val;
9. } rgn_obj_info_t;

```

字段说明：

字段	功能说明
obj_num	目标数量，每个分区只输出检测到的最近目标，最大值为 3
obj[3]	分区对象数组，输出每个分区里检测到的最近目标信息（距离和角度），最大值为 3
range_val	目标的距离信息，单位：mm
angle_val	目标的角度信息

上报场景：需要上报分区检测信息。

3.5 调试命令

本章主要描述 AT6010 的调试命令，用于 AT6010 芯片调试功能。

3.5.1 Register Write

功能说明：

寄存器写命令，支持芯片各模块地址空间内的寄存器值改写。

指令码：0x0

参数：para1: 32-bits register address, para2: 32-bits data to write

发送命令格式（HEX）：58 00 08 Byte0 Byte1 Byte2... Byte7 xx xx

回复帧：参考回复帧格式

e.g. reg_write (0x40003008, 0x0E810F3B)

Send: 58 00 08 08 30 00 40 3B 0F 81 0E B1 01

Resp: 59 00 01 00 5A 00(success). Other: fail.

3.5.2 Register Read

功能说明：

寄存器读命令，支持芯片各模块地址空间内的寄存器值读取。

指令码：0x1

参数：para1: 32-bits register address

发送命令格式（HEX）：58 01 04 Byte0 Byte1 Byte2 Byte3 xx xx

回复帧：参考回复帧格式

e.g. reg_read (0x40003008)

Send: 58 01 04 08 30 00 40 D5 00

Resp: 59 01 04 3B 0F 81 0E 37 01 (value = 0x0E810F3B)

3.5.3 Memory Write

功能说明：

memory 写命令，支持 memory 地址空间内的内容改写。

指令码：0x10

参数：para1: 32-bits memory address, para2: 32-bits data to write

发送命令格式（HEX）：58 10 08 Byte0 Byte1 Byte2... Byte7 xx xx

回复帧：参考回复帧格式

e.g. memory_write (0x20001000, 0xAABBCCDD)

Send: 58 10 08 00 10 00 20 DD CC BB AA AE 03

Resp: 59 10 01 00 6A 00(success). Other: fail.

3.5.4 Memory Read

功能说明：

memory 读命令，支持 memory 地址空间内的内容读取。

指令码：0x11

参数：para1: 32-bits memory address

发送命令格式（HEX）：58 11 04 Byte0 Byte1 Byte2 Byte3 xx xx

回复帧：参考回复帧格式

e.g. memory_read (0x20001000)

Send: 58 11 04 00 10 00 20 9D 00

Resp: 59 11 04 DD CC BB AA 7C 03 (value = 0xAABBCCDD)

3.5.5 Flash Write

功能说明:

flash 写命令, 支持 flash 地址空间内的内容改写。

指令码: 0x14

参数: para1: 32-bits memory address, para2: buf to write (4 <= buf len <= 251)

发送命令格式 (HEX):

head	command	Para len	4 <= buf len <= 251	Check code
0x58	0x14	Size	address and data	Check code

回复帧: 参考回复帧格式

e.g. flash_write(addr, size, buf) // addr=0x0a000000, size=8, buf[8] = {0x11, 0x12, ..., 0x18}

Send: 58 14 0C 00 00 00 0A 11 12 13 14 15 16 17 18 26 01

Resp: 59 14 01 00 6E 00 (success). Other: fail.

4 硬件接口

AT6010 支持 UART / IIC 通信。

4.1 UART 接口

UART 通信按如下配置:

波特率: 921600 bps (默认)

数据位: 8 位

停止位: 1 位

4.2 IIC 接口

AT6010 的 IIC 作为 Slave, 可以配置从器件地址、管脚。参见 target_config.h 文件。

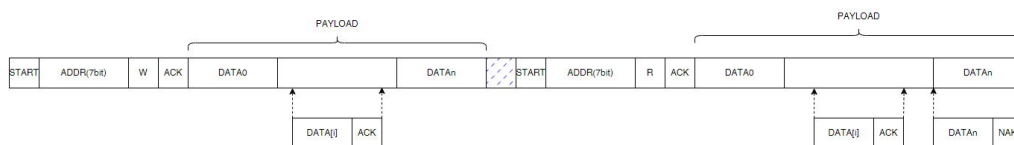
```
#ifndef CFG_CI
#define PARA_CI_PORT          1 /**< 0 -- UART; 1 -- I2C */
#define PARA_CI_I2C_ADDR      0x39U /**< I2C own address */
#define PARA_CI_I2C_SCL_PIN    GPIO_AON_3 /**< GPIO_MCU_6 / GPIO_MCU_8 / GPIO_AON_3*/
#define PARA_CI_I2C_SDA_PIN    GPIO_AON_4 /**< GPIO_MCU_7 / GPIO_MCU_9 / GPIO_AON_4 */
#endif /*! CFG_CI */
```

4.2.1 IIC 时序

CI 通信涉及两个阶段: Master 发送控制帧、Master 读取响应帧。

AT6010 作为 IIC Slave，在第二个阶段发送数据给 Master。

时序图如下所示：



Revision History

Revision	Date	Description
0.1	2022/08/28	Initial Version.
0.2	2023/11/23	Update fmcw det info struct info.
0.3	2023/12/15	Add level probe info and software version.
0.4	2024/02/05	Supplement and improve the documentation instructions.
0.5	2024/03/04	Add the baud rate switching function.
0.6	2024/03/25	Add IIC support.
0.7	2024/05/28	1. Add ULP command. 2. Add auto report command.
0.8	2024/07/02	Add auto report breathing and heart rate.
0.9	2024/07/05	1. Add auto report region information. 2. Modify bhr_det_info_t struct.
1.0	2024/07/23	Modify "Get Software and Hardware Version" ci_ver_struct.